

BNN-YEO: an efficient Bayesian Neural Network for yield estimation and optimization

Zhenxing Dou*

School of Integrated Circuit Science and Engineering,
Beihang University
Beijing, China
zhenxing@buaa.edu.cn

Ming Jia

XC Micro Technologies
Nanjing, China
jia.ming@xcmicro.com

Ming Cheng

Department of Civil, Chemical, Environmental, and
Materials Engineering
University of Bologna
Bologna, Italy
ming.cheng3@unibo.it

Peng Wang[†]

School of Integrated Circuit Science and Engineering,
LMIB, Beihang University
Beijing, China
wang.peng@buaa.edu.cn

ABSTRACT

Yield estimation and optimization is ubiquitous in modern circuit design but remains elusive for large-scale chips. This is largely due to the mounting cost of transistor-level simulation and one's often limited resources. In this study, we propose a novel framework to estimate and optimize yield using Bayesian Neural Network (BNN-YEO). By coupling machine learning method with Bayesian network, our approach can effectively integrate prior knowledge and is unaffected by the overfitting problem prevalent in most surrogate models. With the introduction of a smooth approximation of the indicator function, it incorporates gradient information to facilitate global yield optimization. We examine its effectiveness via numerical experiments on 6T SRAM and found that BNN-YEO provides 100x speedup (in terms of SPICE simulations) over standard Monte Carlo in yield estimation, and 20x faster than the state-of-the-art method for total yield estimation and optimization with improved accuracy.

KEYWORDS

Bayesian Neural Network, yield estimation, yield optimization

ACM Reference Format:

Zhenxing Dou, Ming Cheng, Ming Jia, and Peng Wang. 2024. BNN-YEO: an efficient Bayesian Neural Network for yield estimation and optimization. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3658242>

*First author.

[†]Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3658242>

1 INTRODUCTION

Yield estimation and optimization is ubiquitous in modern circuit design and becoming increasingly paramount with the rising cost of advanced-node chip [5, 11]. At nano-scale, the impacts of process variations such as intra-die mismatches and doping fluctuation are substantial, while aleatory uncertainty from quantum tunnelling further complicates one's effort to maximize yield in the hyperspace of numerous design parameters.

Yield estimation and optimization amounts to the capture of rare events in the distribution tail. Such low probability are traditionally approximated by Monte Carlo-based methods, i.e. via a number of SPICE simulation (Simulation Program with Integrated Circuit Emphasis) [13]. But they becomes forbiddingly expensive as the cost of each simulation soars in tandem with the number of transistors integrated on a chip and are unaffordable with limited computational resources and time. As a result, various sampling techniques have been proposed to reduce the total number of simulations, including Importance Sampling (IS)[7], Quasi-Monte Carlo (Quasi-MC) [9], and Latin Hypercube Sampling (LHS) [10].

Surrogate model is an alternative approach to estimate and optimize yield. By constructing a functional relationship between parameters (inputs) and yield (output), it supplements SPICE simulations and thus reduces the total costs. For example, Polynomial is a popular choice and its polynomial chaos expansion with spectral convergence [4]. Meta-model using Low-Rank Tensor Approximation (LRTA) is another approach which can be solved efficiently with a robust greedy algorithm and calibrated iteratively with an adaptive sampling method [8]. But as the number of random parameters rises, the surrogate approach would inevitably suffer the "Curse of dimensionality", leading to exceedingly high cost. Meanwhile, model distortion may also occur in polynomials of high degree due to the Runge-Kutta phenomenon and overfitting.

In recent years, hybrid framework that combines sampling and surrogate model have become increasingly popular. Yao et al. [14] proposed a rapid SRAM yield analysis method that utilizes both importance sampling and an online surrogate model using adaptive SPICE simulations. In a later study, Wang et al. [11] introduced a heuristic two-stage Adaptive Yield Optimization (AYO) method,

in which Bayesian optimization is employed for optimal design. But its two-stage Monte Carlo introduces meticulous tuning of hyperparameters and thus incurs additional costs. Recently, Yin et al. addressed such issue by proposing a Bayesian yield analysis (BYA) and modified the Gaussian quadrature with the Bernoulli link function [15]. It incorporates an efficient activation learning process by searching for critical regions with extra simulation for each yield estimation. As a result, additional SPICE sampling are needed for yield optimization should the optimization process be too long or the training data set is flawed.

To address the aforementioned challenges in yield estimation and optimization, we propose a Bayesian Neural Network with the following novelty:

- *Bayesian Neural Network as surrogate model.*
To the best of the authors' knowledge, this is the first study incorporating Bayesian Neural Network to estimate and optimize yield.
- *Fast optimization without compromise on yield estimation.*
Smooth approximation facilitates the utilization of gradient information and expedites yield optimization.
- *Overfitting prevention.*
A global surrogate model is constructed so that it alleviates the risk of overfitting prevalent in most surrogate models.
- *Total cost reduction.*
No additional SPICE simulations are needed in optimization once surrogate model training is completed. Based on SRAM, BNN-YEO is 20x faster than the SOTA yield optimization method BYA.

The remaining sections of this paper are organized as follows: In Section 2 we provide a review of the research background and related techniques in yield prediction and optimization. Section 3 outlines our Bayesian Neural Network for yield estimation and optimization. Section 4 illustrates and discusses the experimental results of our approach along with those of the baseline methods, and conclusions are drawn in Section 5.

2 PROBLEM FORMULATION

Denote $\mathbf{x} = [x_1, x_2, \dots, x_{N_x}]^T$ and $\mathbf{v} = [v_1, v_2, \dots, v_{N_v}]^T$ as vectors of size N_x representing design parameters of a circuit and the manufacturing process variation errors, respectively. Uncertainty on the values of the parameters and errors render them as random variables in the related space: $\mathbf{x} \in \mathcal{X}$ and $\mathbf{v} \in \mathcal{V}$. Following standard practice, we assume that the random errors are independent and of standard Gaussian distributions, i.e.,

$$p(\mathbf{v}) = (2\pi)^{-\frac{N_v}{2}} \prod_{j=1}^{N_v} \exp\left(-\frac{(v_j)^2}{2}\right). \quad (1)$$

Let $(\mathbf{x}^{(i)}, \mathbf{v}^{(j)})$ be given values of design parameters and process variations, where the superscripts i and j represents the i -th and j -th sample, respectively. Corresponding circuit performances such as memory read/write time, amplifier gain and etc., $\mathbf{y}^{(i)} = [y_1, y_2, \dots, y_{N_y}]^T$, can be obtained from SPICE simulation:

$$\mathbf{y}^{(i)} = \mathbf{f}(\mathbf{x}^{(i)}, \mathbf{v}^{(j)}), \quad (2)$$

where $\mathbf{f}(\cdot)$ represents the SPICE simulator function. If all performance metrics $\mathbf{y}^{(i)}$ satisfies a pre-defined design specifications $\mathbf{y}^*$, the parameters value $\mathbf{x}^{(i)}$ is considered successful. Otherwise, it is categorized as failure design. This binary result can be represented by a indicator function $I(\mathbf{x}^{(i)}, \mathbf{v}^{(j)})$, in which a value of 1 suggests success design and 0 otherwise. Denoting the probabilistic density function of process variation errors as $p(\mathbf{v})$, one can define circuit yield for the given values design parameters:

$$P_f(\mathbf{x}^{(i)}) = \int_{\mathcal{V}} I(\mathbf{x}^{(i)}, \mathbf{v}) p(\mathbf{v}) d\mathbf{v}, \quad (3)$$

Once yield is estimated, it undergoes optimization to find the optimal values of design parameters $\mathbf{x}^*$:

$$\mathbf{x}^* = \arg \max_{\mathbf{x}} P_f(\mathbf{x}), \quad \text{s.t. } \mathbf{x} \in \mathcal{X}. \quad (4)$$

There are two major challenges in yield design. One is the computational costs of SPICE simulations in estimating yield. For standard practice such as Monte Carlo (MC) method, yield is often approximated by its sample mean,

$$\hat{P}_f(\mathbf{x}^{(i)}) = \frac{1}{M} \sum_{j=1}^M I(\mathbf{x}^{(i)}, \mathbf{v}^{(j)}). \quad (5)$$

where $\{\mathbf{v}^{(j)}\}_{j=1}^M$ is the sample set for process error and generated in accordance with its distribution $p(\mathbf{v})$. As the number of samples increases ($M \rightarrow +\infty$), the yield approximation $\hat{P}_f$ would statistically converge to its true value P_f . However, in practice, failure is often deemed a very small number, slow convergence rate $\mathcal{O}(1/\sqrt{M})$ of Monte Carlo would require a very large number of SPICE simulations and thus incurs significant cost. The second challenge is the lack of analytical form of failure probability ($P_f(\mathbf{x})$) as a function of design parameters. Without it nor its derivative ($\partial P_f / \partial \mathbf{x}$), the optimization process of yield is at best a guess.

3 BNN-YEO FRAMEWORK

Our method, BNN-YEO, consists of two parts. We first employ Bayesian Neural Network to construct a surrogate model for yield with a relatively small number of SPICE simulations. Once such model is trained, it serves as the objective function in optimization process and information on its gradients with respect to design parameter can be utilized for fast convergence.

3.1 Yield Estimation with Bayesian Neural Network

Bayesian Neural Network (BNN) leverages prior knowledge to build a highly accurate surrogate model of circuit performances ($\mathbf{y}$) with a limited number of SPICE simulations[3, 12]:

$$\mathbf{y} \approx \tilde{\mathbf{y}}(\mathbf{x}, \mathbf{v}; \boldsymbol{\theta}), \quad (6)$$

where $\mathbf{x}$ are design parameters, $\mathbf{v}$ denotes random process error and $\boldsymbol{\theta}$ as the model parameters with a prior distribution $P(\boldsymbol{\theta})$.

Training data $\mathcal{D}$ for the BNN model is often from noisy SPICE simulations:

$$\mathcal{D} = \{\mathbf{y}^{(i)}\}_{i=1}^N = \{\mathbf{f}(\mathbf{x}^{(i)}, \mathbf{v}^{(i)}) + \boldsymbol{\epsilon}^{(i)}\}_{i=1}^N, \quad (7)$$

in which N is the size of the train data, errors ϵ can be assumed as independent Gaussian random variables with zero mean and known standard deviation (σ). For the given data $\mathcal{D}$ and prior knowledge, the likelihood of observation can be calculated as:

$$P(\mathcal{D} | \theta) = \prod_{i=1}^{N_y} \prod_{j=1}^N \frac{1}{\sqrt{2\pi\sigma_{i,j}^2}} \exp \left[-\frac{(\tilde{y}_i(\mathbf{x}^{(j)}, \mathbf{v}^{(j)}; \theta) - y_i^{(j)})^2}{2\sigma_{i,j}^2} \right]. \quad (8)$$

Using Bayes' theorem, the posterior likelihood can be obtained:

$$P(\theta | \mathcal{D}) = \frac{P(\mathcal{D} | \theta) P(\theta)}{P(\mathcal{D})} \propto P(\mathcal{D} | \theta) P(\theta). \quad (9)$$

Since calculation for the probability of data set $P(\mathcal{D})$ is analytically intractable, thus in practice, the posterior $P(\theta | \mathcal{D})$ is often not normalized and is expressed with the symbol $\propto$, i.e., an equality up to a constant.

For a fully connected neural network, there are N_l hidden layers in the surrogate model. Its input and output is $\mathbf{x}$ and $\tilde{\mathbf{y}}$, respectively. Its l -th hidden layer is denoted as $\mathbf{z}_l$, ($l = 1, 2, \dots, N_l$):

$$\mathbf{z}_l = \phi(\mathbf{w}_{l-1}\mathbf{z}_{l-1} + \mathbf{b}_{l-1}), \quad \tilde{\mathbf{y}} = \mathbf{w}_{N_l}\mathbf{z}_{N_l} + \mathbf{b}_{N_l}.$$

Here $\mathbf{w}_l$ are the weight matrices and $\mathbf{b}_l$ are the bias vectors. Nonlinear activation function $\phi(\cdot)$ is taken as hyperbolic tangent function in this study. The surrogate model parameters θ are the concatenation of all the weight matrices and bias vectors.

A commonly used prior for θ in Bayesian Neural Networks is that each component of θ follows an independent Gaussian distribution with zero means. For each hidden layer ($l = 1, 2, \dots, N_l$), entries of the weight matrices ($\mathbf{w}_l$) and bias vectors ($\mathbf{b}_l$) are variances $\sigma_{w,l}$ and $\sigma_{b,l}$, respectively. In such case, one can show that as the width of hidden layers goes to infinity with $O(\sqrt{N_l}\sigma_{w,l})$ fixed for $l = 1, 2, \dots, N_l$ [6], prior of the surrogate model becomes a Gaussian process. Since parameters θ in Bayesian Neural Network are drawn from a family of Gaussian variables, it is similar to adding regularization terms to regression models [1]. Therefore, BNN can effectively prevents overfitting in establishing a global surrogate model.

Specific values of model parameters θ are those maximizing the posterior likelihood $P(\theta | \mathcal{D})$. Denoted as θ^* , it can be obtained by numerical optimization for the logarithm of expression (9), using Adam [2] and L-BFGS algorithm [16]. Finally, the posterior circuit performance, $\tilde{\mathbf{y}}(\mathbf{x}, \mathbf{v}, \theta^*)$, can be computed from its surrogate model. Since this step is SPICE-free, one can conduct numerous Monte Carlo on the surrogate model and estimate yield by its sample mean (5).

For an efficient training of the Bayesian Neural Network, Latin Hypercube Sampling and cross-validation [8] are employed. The former is for effective sampling, while the latter determines the number of layers and width in the network structure. In summary, our Bayesian Neural Network for yield is trained as below via Algorithm 1:

3.2 Yield optimization with smooth approximation

Having construct a surrogate model to estimate yield with Bayesian Neural Network (5), its optimization can be expedited if gradient information $\partial p_f / \partial \mathbf{x}$ is known. In other words, one needs to find a

Algorithm 1: Yield Estimation in BNN-YEO.

Input : SPICE sample count N , number of network layers $N_{l,i}$, and width d_i , $i = \{1, 2, 3\}$, number of Monte Carlo realizations M , design parameters values $\mathbf{x}$

Output : Bayesian Neural Network $\mathcal{M}$, yield estimation $\hat{p}_f(\mathbf{x})$

1 Setup.

- Conduct Latin Hypercube Sampling on the design parameter domain and obtain a set of their potential values $\{(\mathbf{x}^{(i)}, \mathbf{v}^{(i)})\}_{i=1}^N$.
- Perform SPICE simulation and obtain the corresponding circuit performances that form the training data set $\mathcal{D}$.

2 Cross-validations.

- Divide the training data set $\mathcal{D}$ into three subsets $\mathcal{V}_i$, $i = \{1, 2, 3\}$. Each is used as a mock validation subset and its corresponding complementary set ($\mathcal{U}_i = \mathcal{D} \setminus \mathcal{V}_i$) serves as the training subset.
- Determine the number of hidden layers N_l and width d . For each of the three training subset, $\mathcal{U}_i$, BNN model is trained for different numbers of hidden layers $N_{l,i}$ and widths d_i . Compute the average error of the surrogate model on the corresponding validation subsets $\mathcal{V}_i$. Select the number of layers N_l and width d for the minimum average error.

3 Model Training.

Train the BBN model of layers N_l and width d with full data set $\mathcal{D}$.

4 Yield Estimation.

Approximate yield from the surrogate model,

$\{\tilde{\mathbf{y}}(\mathbf{x}, \mathbf{v}^{(j)})\}_{j=1}^M$, by performing Monte Carlo sampling of process errors.

$$\hat{p}_f(\mathbf{x}) = \frac{1}{M} \sum_{j=1}^M I(\mathbf{x}, \mathbf{v}^{(j)}).$$

smooth and analytic approximation to the discontinuous indicator function without compromise on the accuracy of yield estimation. This is achieved by introducing the logistic function $H(\cdot)$:

$$H(x) = \frac{1}{2} + \frac{1}{2} \tanh qx. \quad (10)$$

Here q is a user-defined value that corresponds to the sharp transition at $x = 0$. Figure 1 illustrates logistic function at different values of q . One can see that as q increases from 1 to 50, the new function approaches the indicator function. When q tends to infinity, the limit of logistic function (10) is:

$$H(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}. \quad (11)$$

Now for given values of design parameters and process variations, $(\mathbf{x}^{(i)}, \mathbf{v}^{(j)})$, the indicator function can be approximated with

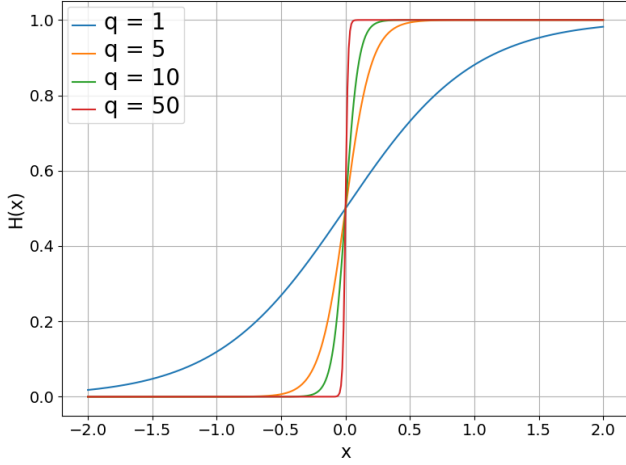


Figure 1: The logistic function (10) at different values of q .

the logistic function and the surrogate model:

$$I(\mathbf{x}^{(i)}, \mathbf{v}^{(j)}) \approx \prod_{k=1}^{N_y} H[y_k^* - \tilde{y}_k(\mathbf{x}^{(i)}, \mathbf{v}^{(j)})]. \quad (12)$$

In turn, its estimation of yield becomes:

$$\tilde{P}_f(\mathbf{x}^{(i)}) = \frac{1}{M} \sum_{j=1}^M \prod_{k=1}^{N_y} H[y_k^* - \tilde{y}_k(\mathbf{x}^{(i)}, \mathbf{v}^{(j)})]. \quad (13)$$

Now we have a smooth and analytical expression of yield ($\tilde{P}_f$). Its gradient information on design parameters can be easily obtained and utilized for its optimization. In our study, Adam algorithm is employed to solve the following optimization problem:

$$\mathbf{x}^* = \arg \max_{\mathbf{x}} \tilde{P}_f(\mathbf{x}) \quad \text{s.t. } \mathbf{x} \in \mathcal{X}, \quad (14)$$

It is noted here that our sampling interval of design parameters must be within the domain $\mathbf{x} \in \mathcal{X}$. However, in practice, Adam optimization may lead to values outside such domain and results in optimization failure. To mitigate such risk, it is necessary to check whether the next-step value $\mathbf{x}'$ surpasses its domain after each iteration. If so, it must be pulled back to the closest boundary value. For instance, for a one-dimensional parameter domain, $\mathcal{X} = [-1, 1]$, if the next optimal point x' is calculated as 1.2, it should be restricted to $x' = 1$. For high dimensions N_x , the aforementioned examination must cover for each parameter. In summary, our yield optimization can be conducted via Algorithm 2:

4 RESULTS AND DISCUSSION

We examine the performance of our BNN-YEO via 6T-SRAM with comparison to one state-of-the-art method, Bayesian Yield Analysis (BYA) [15]. To be specific, their performances on yield estimation, optimization are studied, respectively. Without notification otherwise, the hyperparameter in the logistic function is taken as $q = 50$. Maximum number of layers and width in the Bayesian Neural Network is set as 7 and 64, respectively. All experiments were conducted on a Linux server with 500GB of memory, one

Algorithm 2: Yield optimization in BNN-YEO

Input : The trained Bayesian neural network $\mathcal{M}$, number of optimization iterations N_o , and a threshold value η for accuracy.

Output: Optimal design parameter $\mathbf{x}^*$.

- 1 **for** $i = 1$ **to** N_o **do**
- 2 Search for potential values of optimal design parameters, $\mathbf{x}^*$, by solving the optimization problem (14) using Adam;
- 3 Check if $\mathbf{x}^*$ is in the defined domain $\mathcal{X}$. If not, round it to the closest boundary values;
- 4 Update and examine the corresponding yield ($\tilde{p}_f$) using the surrogate model based on Bayesian neural Network (13);
- 5 If the difference between the new yield $\tilde{p}_f$ and the target value p_f^* is smaller than the threshold value η , terminate;
- 6 **end**

AMD EPYC 7402 24-core processor, and one Nvidia GeForce RTX 3080 Ti graphics card.

4.1 Yield Estimation

The 6T-SRAM is a canonical test case for yield estimation and optimization [14, 15]. Its simplified schematic diagram is shown in Fig. 2. In our study, write delay is taken as the circuit output performance (y), and gate oxide width and length under a 45nm CMOS process for all six transistors are the design parameters of interests ($\mathbf{x}$). In total, there are $N_x = 12$ design parameters for the 6T-SRAM. The circuit is performing within 27 degrees Celsius. The write delay for SRAM is the time taken from the start of the write operation to reaching 90% of the threshold voltage.

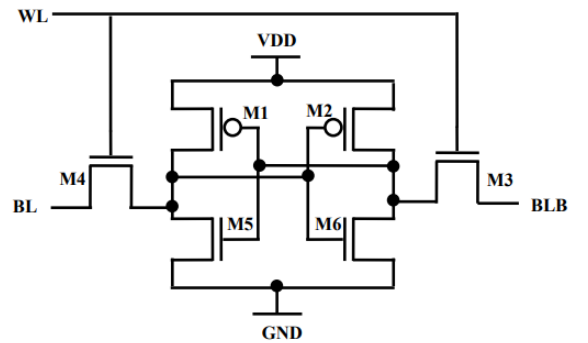


Figure 2: Simplified schematic for 6T-SRAM.

We now examine the accuracy of the BNN-YEO algorithm in yield estimation. The target yield is set as $P_f = 99.9\%$, which is achieved with 50,000 Monte Carlo SPICE simulations. Ten experiments are repeated for BNN-YEO and BYA, respectively, to reduce the impacts of random seeds on yield estimation. The logarithm of their mean absolute errors with respect to target yield are shown

in Fig. 3 for various number of SPICE simulations. It is clear that for each method, its estimation accuracy improves as simulations increases. Comparing to BYA, BNN-YEO exhibits a sharper drop in error at the very beginning to almost $O(10^{-4})$ and then follows similar pattern at later stage with more simulations. In general, its mean error for yield estimation is an order of magnitude lower than that of BYA.

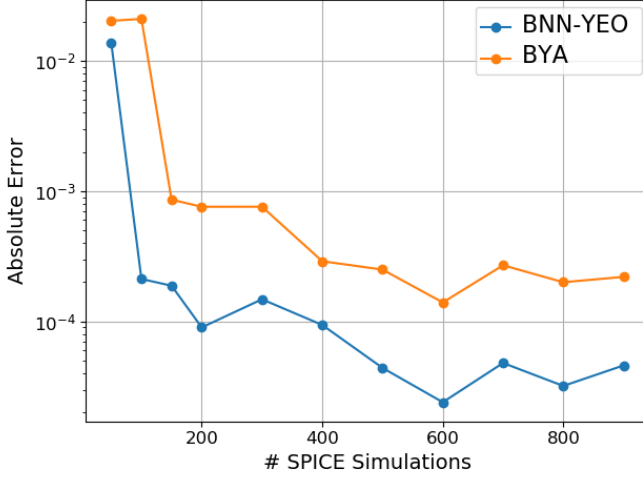


Figure 3: Yield Estimation Absolute Error of 6T-SRAM

For a closer look, we present the absolute errors in yield estimation for both BYA and BNN-YEO in Table 1. Their best, worse and mean results of errors are included for 100, 200, 300 and 500 SPICE simulations, respectively. For the same number of SPICE samples, BNN-YEO generally provides an-order-of-magnitude improvements in yield estimation than BYA. For the same order of accuracy, all three cases of measurements in BNN-YEO reach an absolute error of $O(10^{-4})$ at just 100 simulations and settle to $O(10^{-5})$ with 500 samples. In contrast, it takes BYA 500 SPICE simulations to converge at $O(10^{-4})$. We thus conclude that for yield estimation, BNN-YEO is 5X faster than BYA and 100x more efficient than standard Monte Carlo in terms of SPICE simulations required.

4.2 Yield Optimization

Let us now investigate the effectiveness of BNN-YEO in yield optimization. For comparison, results from BYA method are also included in Table 2. The best, worst and mean results of ten experiments from each method are obtained using 500, 5000 and 10,000 SPICE simulations, respectively. One can see that both approaches improve their optimizations with more simulations involved. For BNN-YEO, all its three cases achieve near-optimal values of 99.98% using just 500 samples. Though they can be further elevated with 5000 and 10,000 simulations, the improvements of 0.01% are somehow limited. This is because that once BNN-YEO completes model training in the yield estimation phase, its optimization is performed on its surrogate model ($\hat{P}_f$) and thus no more simulations are required. For BYA, its optimization performances at small number of SPICE simulations are at best mediocre. With 500 samples, its yield

optimization results are below 50%. It increases to $< 80\%$ for 5000 simulations and reaches 99% using 10,000 samples. Such performance may be due to its adaptive nature with a continuous inflow of data. It would be discussed and analyzed in details in subsequent subsection. After all, BNN-YEO shows a 20x improvements that state-of-the-art method in SPICE simulations costs.

4.3 Computational Cost Analysis

We tabulate the computational costs for Monte Carlo, BNN-YEO and BYA, respectively, in Table 3. It takes roughly 8.65 seconds to run one SPICE simulations on the current computer environment. For Monte Carlo, 50,000 simulations are needed to converge to the target yield value of 99.9%. For BNN-YEO, its surrogate model requires 500 simulations to train and takes only 0.0032 seconds for one execution. This is 2700x faster than SPICE simulations. Such improvement significantly expedites the yield prediction process in BNN-YEO. For BYA, its Gaussian Process model is also cheaper than SPICE simulation, e.g. 0.126 seconds per run. But its adaptive update on the surrogate model in optimization often entails a continuous inflow of SPICE simulations, which eats up its gain in model speed.

Table 3: Computational Costs of Monte Carlo (MC), BNN-YEO and BYA.

	MC	BYA	BNN-YEO
Simulation Speed	8.65s	0.126s	0.0032s
# Simulation	50,000	10,000	500

For better understanding, we now conduct detailed analysis on the computational expenses. This is done through a simple white-box model as circuit response:

$$f(\mathbf{x}, \mathbf{v}) = \sum_{i=1}^{N_x} (x_i + v_i)^2, \quad \mathbf{x} \in [-1, 1]^{N_x}. \quad (15)$$

Here Success is defined if $f(\mathbf{x}, \mathbf{v}) \leq N_x$. In other words, the optimal design parameters values for highest yield are: $\mathbf{x}^* = [0, 0, \dots, 0]^T$.

Table 4 presents the optimized yield results from BYA and BNN-YEO for three cases, namely, $N_x = 4$, $N_x = 10$ and $N_x = 18$. For both methods, 100, 300 and 500 simulations of (15) are used to construct the surrogate models in each case. Their optimizations all initiate at the same values, $\mathbf{x}_0 = [-1, -1, \dots, -1]^T$. The target yield values are obtained via 100,000 simulations and are also listed. To reduce the impacts of random seeds on training data, 10 experiments are repeated for each method at each case. Their best, worst and mean performance are also included. It is evident in Table 4, performance of BYA deteriorates as parameter dimension increases. At $N_x = 10$ and $N_x = 18$, BYA cannot find the optimal solution using its surrogate model from 500 samples of (15), but needs 1000 and 3000 samples, respectively. On the other hand, BNN-YEO is capable to capture target yield with only 500 simulations. Such contrast in performance lies in the selection of surrogate models. BYA employs a Gaussian process to construct a surrogate model, which is prone to overfitting and has certain limitations. It can provide a good estimate of optimal value should the iteration value be included in its

Table 1: Yield analysis absolute error for 6T-SRAM

		#Simulation=100		#Simulation=200		#Simulation=300		#Simulation=500	
Test		BYA	BNN-YEO	BYA	BNN-YEO	BYA	BNN-YEO	BYA	BNN-YEO
6T-SRAM (Yield=99.9 %)	Best	1.95E-03	1.38E-04	3.78E-05	6.77E-05	4.11E-05	3.26E-05	1.75E-04	2.62E-05
	Mean	2.13E-02	2.12E-04	7.59E-04	9.00E-05	7.61E-04	9.42E-05	2.93E-04	4.45E-05
	Worst	4.56E-02	5.76E-04	1.52E-03	1.35E-04	9.63E-04	1.20E-04	5.12E-04	7.40E-05

Table 2: Yield optimization for 6T-SRAM

		#Simulation=500		#Simulation=5000		#Simulation=10,000	
Test		BYA	BNN-YEO	BYA	BNN-YEO	BYA	BNN-YEO
6T-SRAM	Best	45.45 %	99.98%	75.59 %	99.99%	99.88 %	99.99%
	Mean	42.34 %	99.92%	67.54%	99.96%	99.68 %	99.98%
	Worst	38.59 %	99.87%	61.37%	99.94%	98.52 %	99.98%

training set. Otherwise, incorrect estimation may ensue and Gaussian process has to be continuously updated through resampling of new SPICE simulations. Our Bayesian neural network is instead sampled from a family of Gaussian distributions for optimal model parameters. Therefore, it can construct a global surrogate model avoiding model distortion and significantly reduce simulation cost.

Table 4: Yield Estimation and Optimization for Case (15).

Test		BYA	BNN-YEO
$N_x = 4$ (Target yield 59.44%)	Best	59.41%	59.42%
	Worst	59.27%	59.37%
	Mean	59.35%	59.39%
$N_x = 10$ (Target yield 55.96%)	Best	50.61%	55.93%
	Worst	46.41%	55.87%
	Mean	48.24%	55.89%
$N_x = 18$ (Target yield 54.44%)	Best	44.88%	54.39%
	Worst	35.87%	54.06%
	Mean	40.90%	54.24%

5 CONCLUSION

In this paper, a novel framework (BNN-YEO) for yield estimation and optimization is proposed. Based on Bayesian Neural Network, it constructs a surrogate model with prior information and replaces SPICE simulations in yield optimization process. Its verification on 6T-SRAM led to the following conclusions:

- The new method provides accurate results in yield estimation and optimization with much fewer SPICE simulations than Monte Carlo and state-of-the-art BYA method.
- Its surrogate model is effective for optimization and at least 2000 times faster than SPICE simulation.
- The globally optimal parameters of the surrogate model are obtained by solving from a family of Gaussian distributions, thereby helping alleviate the common risk of overfitting in many surrogate models.
- Performance of the BNN-YEO approach may be further enhanced if its optimal structure of the Bayesian Neural Network can be found, possibly by another form of machine learning such as reinforced learning.

6 ACKNOWLEDGMENTS

This work is supported by The National Key Research and Development Program of China under Grants 2022YFB4400402.

REFERENCES

- [1] Peter J Bickel, Bo Li, Alexandre B Tsybakov, Sara A van de Geer, Bin Yu, Teófilo Valdés, Carlos Rivero, Jianqing Fan, and Aad van der Vaart. 2006. Regularization in statistics. *Test* 15 (2006), 271–344.
- [2] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [3] Igor Kononenko. 1989. Bayesian neural networks. *Biol. Cybern.* 61, 5 (1989), 361–370.
- [4] Jing Li and Dongbin Xiu. 2010. Evaluation of failure probability via surrogate models. *J. Comput. Phys.* 229, 23 (2010), 8966–8980.
- [5] Bo Liu, Francisco V Fernández, and Georges GE Gielen. 2011. Efficient and accurate statistical analog yield optimization and variation-aware circuit sizing based on computational intelligence techniques. *IEEE TCAD* 30, 6 (2011), 793–805.
- [6] Radford M Neal. 2012. *Bayesian learning for neural networks*. Vol. 118. Springer Science & Business Media.
- [7] Masood Qazi, Mehul Tikekar, Lara Dolecek, Devavrat Shah, and Anantha Chandrakasan. 2010. Loop flattening & spherical sampling: Highly efficient model reduction techniques for SRAM yield analysis. In *Proc. of DATE 2010*. IEEE, 801–806.
- [8] Xiao Shi, Hao Yan, Qiancun Huang, Jiajia Zhang, Longxing Shi, and Lei He. 2019. Meta-model based high-dimensional yield analysis using low-rank tensor approximation. In *Proc. of the 56th DAC 2019*. 1–6.
- [9] Amith Singhee, Sonia Singhal, and Rob A Rutenbar. 2008. Practical, fast Monte Carlo statistical static timing analysis: Why and how. In *Proc. of ICCAD 2008*. IEEE, 190–195.
- [10] Michael Stein. 1987. Large sample properties of simulations using Latin hypercube sampling. *Technometrics* 29 (1987), 143–151.
- [11] Mengshuo Wang, Wenlong Lv, Fan Yang, Changhao Yan, Wei Cai, Dian Zhou, and Xuan Zeng. 2017. Efficient yield optimization for analog and SRAM circuits via Gaussian process regression and adaptive yield estimation. *IEEE TCAD* 37, 10 (2017), 1929–1942.
- [12] Liu Yang, Xuhui Meng, and George Em Karniadakis. 2021. B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. *J. Comput. Phys.* 425 (2021), 109913.
- [13] Jian Yao, Zuochang Ye, and Yan Wang. 2012. Statistical analysis of process variations in RF/mm-wave circuits with on-the-fly passive macro-modeling. *IEEE Trans. Microw. Theory Techn.* 61, 2 (2012), 727–735.
- [14] Jian Yao, Zuochang Ye, and Yan Wang. 2014. An efficient SRAM yield analysis and optimization method with adaptive online surrogate modeling. *IEEE TVLSI* 23, 7 (2014), 1245–1253.
- [15] Shuo Yin, Xiang Jin, Linxu Shi, Kang Wang, and Wei W Xing. 2022. Efficient bayesian yield analysis and optimization with active learning. In *Proc. of the 59th DAC 2022*. 1195–1200.
- [16] Ciyou Zhu, Richard H Byrd, Peihuang Lu, and Jorge Nocedal. 1997. Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization. *TOMS* 23, 4 (1997), 550–560.

Enabling Multiple Tensor-wise Operator Fusion for Transformer Models on Spatial Accelerators

Lei Xu, Zhiwen Mo, Qin Wang, Jianfei Jiang, Naifeng Jing*
Shanghai Jiao Tong University, Shanghai, China
xlxl1027@sjtu.edu.cn, sjtuj@sjtu.edu.cn

ABSTRACT

In transformer models, data reuse within an operator is insufficient, which prompts more aggressive multiple tensor-wise operator fusion (*multi-tensor fusion*). Due to the complexity in tensor-wise operator dataflow, conventional fusion techniques often fall short by limited dataflow options and short fusion length. In this study, we first identify three challenges on multi-tensor fusion that result in inferior fusions. Then we propose dataflow adaptive tiling (DAT), a novel inter-operator dataflow to enable an efficient fusion of multiple operators connected in any form and chained in any length. Then, we broaden the dataflow exploration from intra-operator to inter-operator and develop an exploration framework to quickly find the best dataflow on spatial accelerators with given on-chip buffer size. Experiment results show that DAT delivers $2.24\times$ and $1.74\times$ speedup and 35.5% and 15.5% energy savings on average for edge and cloud accelerators, respectively, comparing to the state-of-the-art dataflow explorer FLAT. DAT is open-sourced at <https://github.com/lxu28973/DAT.git>.

KEYWORDS

dataflow, inter-operator, fusion, accelerator, transformer

1 INTRODUCTION

The proliferation of transformer models has brought a revolution in the realm of machine learning, and the potential of transformer models stems from their fundamental attention mechanism [17]. Fig. 1 visualizes a typical multi-head attention layer, highlighting three primary operator types, i.e. tensor-wise operators (matrix multiplication), row-wise operators (softmax), and point-wise operators (addition), which present distinct compute and memory characteristics. For example, the tensor-wise matrix multiplication (*mm*) operator requires intensive MAC computations particular in $Q \times K^T$ with large sequence length.

For acceleration, various spatial accelerator architectures have been proposed [2, 5, 8, 9]. They often equip a number of processing engines (PEs) and apply specific dataflow to schedule the computation and memory access on these PEs over time and space. For the best performance, prior researches have studied different dataflows

*Naifeng Jing is the corresponding author. This work is supported by the National Natural Science Foundation of China (No. 92264108).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3657317>

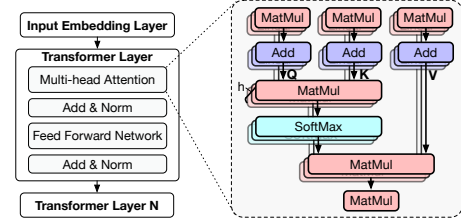


Figure 1: A typical multi-head attention layer in transformer.

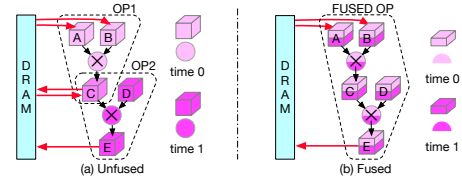


Figure 2: Multi-operator fusion further reduces off-chip memory traffic than unfused operators op_1 and op_2 .

to exploit data reuse within a single tensor-wise operator, namely intra-operator dataflow, particular in convolution neural networks (CNNs) [7, 12, 18, 19]. However, $Q \times K^T$, which dominates the computation in transformer when facing long sequence, presents insufficient data reuse because there is no reuse at the output channel and batch dimensions as in CNNs. Therefore, it poses notable stress on off-chip memory bandwidth as reported in [3, 10, 20].

For optimization, some researches have proposed operator fusion, or inter-operator dataflow [5, 10]. Compared to intra-operator which leaves the intermediate data repeatedly read from and written to memory across an operator boundary as in Fig. 2(a), inter-operator dataflow exploits data reuse between operators. If it can ideally fuse all the operators, the memory traffic on intermediate data can be totally eliminated as in Fig. 2(b). However as in Sec. 3.1, we find that multiple tensor-wise operators fusion (*multi-tensor fusion*) is non-trivial because:

- Contradictive dataflows between two tensor-wise operators prohibit multi-tensor fusion.
- Obstacle operator (e.g. softmax) between two tensor-wise operators prohibits their fusion.
- Directly fusing two compatible tensor-wise operators may not be globally optimal, although each operator bears an optimal intra-operator dataflow.

These challenges confine the conventional fusion techniques to two operators with only a few fixed dataflow options, leaving much design space unexplored for multi-tensor fusion. Our study first addresses these challenges by proposing a dataflow adaptive tiling (DAT). It adapts the tile schedule and size upon fusion, so that any dataflow in a pair of operators can be fused together by eliminating the contradiction or obstacle. Based on DAT fusing two operators, more operators using different dataflows connected in

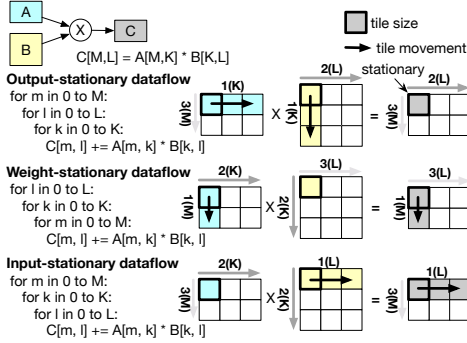


Figure 3: Three intra-operator dataflows for operator $A \times B = C$. We use term $X(D)$ to denote the scheduling by loop order “X” on dimension “D” as in the left code. For example in OS, “1(K)” implies the innermost loop on dimension K.

different forms can be fused, so that we can examine more dataflow options in the design space for an optimal one. As far as we know, DAT is the first to enable multi-tensor fusion in an arbitrary way rather than previous fusion using fixed dataflow options. And it can fuse other tensor-wise operators not limited to attention layer.

In fact, our proposed DAT opens broader search space for multi-tensor fusion. For fast exploration, we then propose a framework built upon conventional intra-operator dataflow exploration. It decouples the search on scheduling, tiling and mapping, and can analytically find the optimal solution in a faster way.

At last, we apply DAT on attention-based models, including BERT, GPT-2, XLM, Blenderbot, DeBERTa-v2, LLaMA2, and ALBERT, which consists of different sized attention operators. The experiment results show that for typical spatial accelerators on edge and cloud, DAT delivers 2.24× and 1.74× speedup and 35.5% and 15.5% energy savings on average, respectively comparing with state-of-the-art dataflow explorer FLAT [10]. Furthermore, DAT works better on long input sequence (i.e. 64K tokens) without increasing the on-chip buffer size and off-chip memory bandwidth.

2 PREVIOUS WORKS AND BACKGROUND

2.1 Spatial Accelerator

As sketched in Fig. 9, spatial accelerators are primarily structured around processing engines (PEs). Each PE equips a number of MAC units, a local buffer and dedicated interconnections among MACs to facilitate data reuse. In this paper, we assume the most common three-level memory hierarchy, i.e. local buffer, on-chip buffer and off-chip memory as in many spatial accelerators [1, 2, 5, 8, 9, 15].

2.2 Intra- and Inter-Operator Dataflow

Efficiently running DNN and transformer models on spatial accelerator heavily relies on the dataflow, which mainly consists of three key decisions, i.e. scheduling, tiling and mapping. While mapping decides how the MAC units compute the tiled operator and therefore decides the parallelism and data reuse at local buffer level, scheduling and tiling decide how data are reused at the on-chip buffer level. There have been many dataflow studies focusing on convolutions on spatial accelerator [7, 11, 12, 18, 19]. In short, they primarily target on intra-operator dataflow to reduce the off-chip memory traffic with given on-chip buffer size.

Fig. 3 shows three most common intra-operator dataflow options, i.e. output stationary (OS), weight stationary (WS) and input stationary (IS) for an operator $A_{MK} \times B_{KL} = C_{ML}$. First, matrix A, B and C are split into smaller tiles to ensure one tile from each of the three matrices can fit into the on-chip buffer. Regarding the tile scheduling, OS first traverses dimension K as implied by the code innermost loop “1(K)”. So C_{ML} , which does not have dimension K, is stationary and reusable. By carefully tuning the tile scheduling and size, spatial accelerator can effectively exploit data reuse to reduce the off-chip memory traffic for a memory-bound operator.

However due to the low operational intensity, exploiting the intra-operator dataflow alone is insufficient in attention layer [3, 10, 20]. As a result, the need of inter-operator dataflow for multi-tensor fusion, arises for keeping intermediate tile data on-chip as much as possible to further reduce off-chip memory traffic. Table 1 briefly summarizes the SOTA dataflow explorers. In short, due to the three challenges, they only fuse point-wise operators [14, 16] or two tensor-wise operators with a fixed dataflow [5, 10]. Later we will see that the lack of holistical search in the dataflow design space will result in local optimization on multi-tensor fusion.

3 DATAFLOW ADAPTIVE TILING ON FUSING TENSOR-WISE OPERATORS

3.1 Three Challenges on Multi-Tensor Fusion

Contradictive dataflow prohibits fusion. Fig. 3 has illustrated the intra-operator dataflow. When we tend to fuse two operators together, the two dataflows chosen by each operator may be contradictive on the binding tensor, because the two dataflows may require the tile in the binding tensor to traverse in different orders, or to be split into different sizes.

Consider two operators mm_1 and mm_2 connecting five tensors calculated in sequence $A_{MK} \times B_{KL} = C_{ML}$ and $C_{ML} \times D_{LN} = E_{MN}$ as in Fig. 4(a). Assume mm_1 and mm_2 adopt IS and OS respectively. Fig. 4(b) shows that the dataflow is contradictive in the binding tensor C_{ML} . To be specific, because dimension L traverses before K with IS in mm_1 , the tile $\textcircled{C}$ is calculated as a partial result. While with OS in mm_2 , it requires $\textcircled{C}$ as a final result so that dimension L can traverse first. In other words, $\textcircled{C}$ in mm_2 cannot start calculation when mm_1 only gives a partial result. In addition, the tile sizes may also be different. So it is impossible to directly fuse them.

Prior studies tackle this problem by fusing two operators using compatible dataflows. For example, TANGRAM[5] proposes OS-IS fusion with the same tile size as shown in Fig. 4(c). With OS in mm_1 , it first traverses dimension K to calculate a tile $\textcircled{C}$ in C_{ML} , which is stationary and serves mm_2 with IS to calculate the partial result tile $\textcircled{C}$ in E_{MN} . Unfortunately, it only works on fusing two operators with compatible dataflow options. When attempting a third operator mm_3 , contradiction still arises because a partial result tile $\textcircled{C}$ in mm_2 cannot serve mm_3 to proceed the calculation.

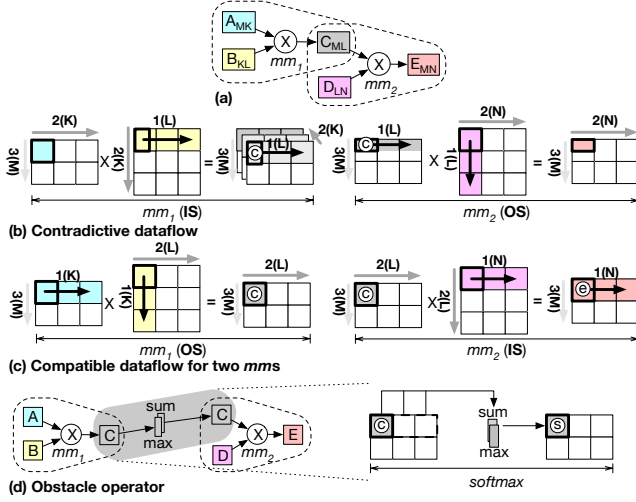
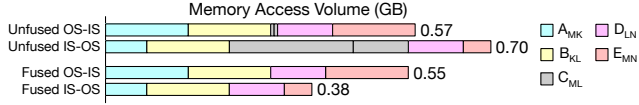
Some other studies proposes other fixed dataflow combinations on fusing two operators, e.g. OS-OS [1, 10]. However, it is obvious that the limited number of fixed dataflow options prohibits the design space exploration for an optimal multi-operator fusion.

Obstacle operator prohibits fusion. Then, we find that the presence of row-wise operators, such as *softmax* and *reduce*, prevent operator fusion. We call it obstacle operator.

Fig. 4(d) illustrates how *softmax* acts as an obstacle in the attention layer, where the preceding mm_1 and following mm_2 cannot be

Table 1: Summary of the SOTA dataflow explorers.

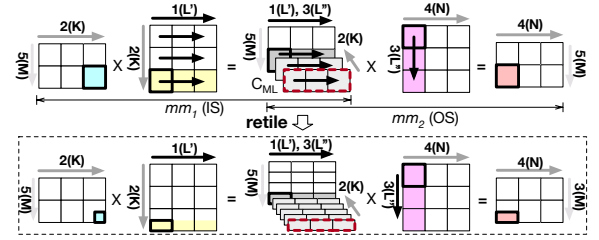
Features	Intra-operator	Inter-operator			
	CoSA, ... [7, 12, 18, 19]	DNNFusion, ... [14, 16]	TANGRAM [5]	Flat[10]	DAT (This Work)
Tensor-wise operator fusion scheme	not discussed	do not fuse	OS-IS fusion	OS-IS fusion	OS-IS fusion
Inter-operator schedule exploration	×	×	×	×	✓
Tiling size optimization scheme	optimize for intra-operator	optimize for intra-operator	fixed tiling method	fixed tiling method	adaptive tiling method
Row-wise operator fusion scheme	not discussed	do not fuse	not discussed	row-level fusion	tile-level fusion

**Figure 4: Contradictive dataflow and obstacle operator challenge multi-operator fusion.****Figure 5: Off-chip memory access volume for unfused OS-IS, IS-IS and fused OS-IS, IS-IS.**

fused. This is because with only one tile © available in C_{ML} , the subsequent operator cannot proceed since *softmax* algorithm in Eq. (1) needs to compute the maximum and sum of the entire row in C_{ML} to generate one result tile ©. In other words, with only result available in ©, mm_2 cannot proceed to enable fusion.

Directly fusing two operators may not be globally optimal. Even if two operators can be fused together, directly fusing them may not be globally optimal in terms of off-chip memory traffic given the on-chip buffer size. Continuing with the example in Fig. 4(a) using configurations from BERT[4], where $mm_1 : A_{512,768} \times B_{768,64} = C_{512,64}$ and $mm_2 : C_{512,64} \times D_{64,512} = E_{512,512}$. The head number and batch size are set to 12 and 16, and the on-chip buffer size is 512KB. As shown in the statistics in Fig. 5, in the unfused situation, the OS-IS dataflow is preferred. In mm_1 , the reading of A_{MK} and B_{KL} requires 0.3GB, and writing to C_{ML} necessitates 0.01GB. In mm_2 , reading C_{ML} requires 0.01GB, while accessing D_{LN} and E_{MN} demands 0.25GB. This leads to a total memory access volume of 0.57GB. Upon direct fusion using IS-IS, the total memory access volume decreases to 0.55GB.

When attempting to fuse them using IS-IS with DAT, despite IS-IS requiring 0.7GB (23% more than OS-IS) in their unfused versions, the absence of intermediate data access significantly alters the scenario. Consequently, the fused IS-IS dataflow necessitates only a total volume of 0.38GB, reducing memory traffic by 31%

**Figure 6: Two steps of DAT, i.e. re-scheduling and re-tiling, to eliminate the contradictive dataflows.**

compared to directly fusing the two operators using OS-IS. This reduction could substantially benefit the overall performance in a memory-bound system.

3.2 DAT Eliminating the Contradictive Dataflow

This section will introduce how DAT eliminates the contradictive dataflow to enable multi-tensor fusion with arbitrary dataflows in any length. In short, DAT apply two steps. First, it fuses contradictive dataflows by re-scheduling the contradictive dimensions, and then re-tiling the tiles for efficient buffer usage.

Fig. 6 still uses the example in Fig. 4(b) to illustrate how DAT eliminates the contradiction between mm_1 and mm_2 with IS-IS fusion. Originally as discussed in Sec. 3.1, it is the partial results in dimension L calculated in mm_1 with IS that prevents fusion. So DAT should calculate the final results as soon as possible to enable the fusion. To this goal, DAT traverses dimension L in two ways, i.e. L' for mm_1 and L'' for mm_2 , to smooth the dataflow on C_{ML} . The traversal on L' in mm_1 generates partial results, followed by traversal on dimension K in mm_1 for final results in C_{ML} . Then, the traversal on L'' in mm_2 follows traversal on K in mm_1 , enabling the final results generated in mm_1 can be reused immediately for mm_2 . By this re-scheduling, DAT succeeds in sticking to IS-IS.

Next, to accommodate the intermediate tiles in C_{ML} still in the on-chip buffer, DAT re-tiles the tile shape both in mm_1 and mm_2 . Specifically as illustrated at the bottom in Fig. 6, it reduces the tile in both height and width to make the intermediate tiles smaller enough. By a careful re-tiling calculation as in Sec. 4.2, a reduced number of C_{ML} rows are stored on-chip without exceeding the buffer size, and yet the increased accesses on C_{ML} will not result in additional off-chip memory access.

The re-scheduling and re-tiling in DAT also work for other dataflow options, e.g. re-schedule the dimension M and re-tiling in WS-IS fusion. It is important to note that by fusing two operators with arbitrary dataflows, DAT can further extend to more operators connected in any form, and fuse them to an adequate length in terms of memory traffic. By this chaining, DAT enables us to examine all the dataflow combinations rather than a few fixed options for an optimal fusion.

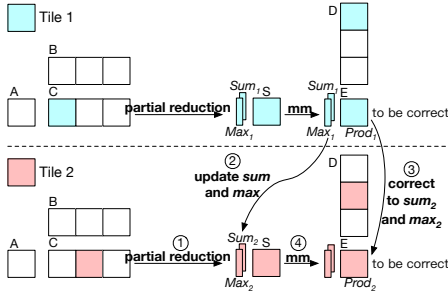


Figure 7: DAT tiles the row-wise operator to enable fusion.

3.3 DAT Eliminating the Obstacle Operator

From the observation in Section 3.1, we find that the key to eliminate the obstacle operator impact is to enable tiling on row-wise operators. In other words, we are able to use a tile of data to start next operator's computation instead of waiting for the whole row.

We use *softmax* as an example to demonstrate how DAT eliminates the obstacle operator. The *softmax* is defined in Eq. (1), involving two row-wise operations that compute the maximum ($\max = \max(x_0, \dots, x_n)$) and sum ($\text{sum} = e^{x_0 - \max} + \dots + e^{x_n - \max}$).

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}} = \frac{e^{x_i - x_{\max}}}{\sum_{j=1}^N e^{x_j - x_{\max}}} \quad (1)$$

As shown in Fig. 7, DAT employs several steps, i.e. partial computation, update, correction and multiplication to eliminate the obstacle *softmax*(C) between $A \times B = C$ and $C \times D = E$. Starting with tile 1, DAT computes the partial maximum ($\max_1 = \max(x_0, \dots, x_{n1})$) and partial sum ($\text{sum}_1 = e^{x_0 - \max_1} + \dots + e^{x_{n1} - \max_1}$) for the first tile and stores them on-chip. Then, it utilizes these temporary maximum and sum values to compute the *softmax* and subsequently performs matrix multiplication to obtain the result *prod*₁.

For tile 2, DAT 1) computes the maximum ($\max_2$) and sum (sum_2) for tile 2, 2) updates the $\max_2$ with the old $\max_1$ ($\max_2 = \max(\max_1, \max_2)$) and computes the new sum ($\text{sum}_2 = \text{sum}_1 \times e^{\max_1 - \max_2} + \text{sum}_2$), 3) uses the new $\max_2$ and sum_2 to correct *prod*₁ ($\text{prod}_1 = \text{prod}_1 \times e^{\max_1 - \max_2} \times \frac{\text{sum}_1}{\text{sum}_2}$), 4) performs the matrix multiplication and add *prod*₁ to obtain *prod*₂.

This process proceeds for the remaining tiles. Eventually when computing the last tile, $\max_{\text{last}}$ and sum_{last} represent the overall maximum and sum of the entire row, ensuring correctness.

This scheme applies to other row-wise operators such as row-wise *reduction* by enabling tiling on row-wise operators, therefore enables their fusion with tensor-wise operator. After fusion, it significantly benefits the off-chip memory traffic. Although there is some overhead, including small on-chip buffer size requirement for the *max* and *sum*, and some recomputation on *prod*, their costs are negligible when consider the benefits on memory traffic.

4 DAT EXPLORATION FRAMEWORK

DAT enables all intra-operator dataflow combinations upon multi-tensor fusion. Base on DAT, we broaden the dataflow exploration from intra-operator to inter-operator. For faster exploration, we propose three decoupled optimization steps, i.e. scheduling, tiling, and PE hardware mapping, as illustrated in Fig. 8.

On top of the intra-operator dataflow, which determines the tile scheduling, tile size, and tile mapping within an operator, DAT also considers tile scheduling among multiple operators. To represent

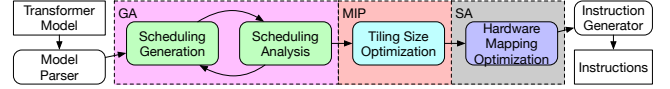


Figure 8: DAT exploration framework with decoupled steps.

inter-operator scheduling, we apply a graph structure where a node denotes an operator and an edge indicates a binding tensor between two operators. For example, in Fig. 4(a), mm_1 and mm_2 are nodes, and the binding tensor C_{ML} forms an edge between them. We introduce a *fusion* attribute on the edge to determine whether the two operators will be fused together. When set to true, the consumer operator (mm_2) should initiate computation as soon as the required tiles in the binding tensor (C_{ML}) are ready. Additionally, we add a *priority* attribute to the node to determine the execution order of operators in the graph when there are multiple executable operators, with higher priority executed first. Regarding intra-operator dataflow annotations, we follow a similar approach to the intra-operator dataflow explorer [12].

We choose an operator, not a tensor, as a node in our graph because DAT may create different dataflows for the binding tensor in two unfused or contradictive operators. As in Fig. 6, the traversal on L' and L'' results in distinct dataflows for the binding tensor C_{ML} in mm_1 and mm_2 . This allows for two different tiling on the same tensor. By representing the operator as a node, the binding tensor can have different attributes in two connected operators.

4.1 Scheduling Generation and Analysis

Different scheduling of operators heavily impact the DAT results. To examine all the combinations, we apply a genetic algorithm to generate different schedule options and an analytical method to select superior ones. It significantly differs from conventional exploration because it decouples with the tile size optimization (next step), and the analysis is optimized to be fast without requiring time-consuming profiling or simulations used in [16, 19].

Still taking the example in Fig. 4(a), our genetic algorithm works as follows:

1. Initialization. In the DAT graph, we randomly select edges for fusion, randomly assign priorities and dimension orders to nodes. Among these assignments, the priorities of nodes are constrained by their dependencies. For example, in Fig. 4(a), we set the edge on tensor C_{ML} as *fusion*, specify that mm_1 should execute before mm_2 , and define the dimension order as IS (1L, 2K, 3M) for mm_1 and OS (1L, 2N, 3M) for mm_2 , to initialize a schedule as in Fig. 4(b).

2. Evolution. We introduce mutations in edge *fusion*, node *priority* and dimension order. Crossovers are performed by randomly selecting two individuals and exchanging nodes' intra-operator schedules within parts of their graphs. For example, the individuals in Fig. 4 (b) and (c) are selected, with schedule (IS-OS) and (OS-IS) respectively, and their mm_2 parts are exchanged, resulting two new individuals with schedule (IS-IS) and (OS-OS) respectively.

3. Analyze and Filter. With generated schedules, the memory access volume (MA) and buffer size requirements (BR) can be symbolically analyzed. We extend the intra-operator analysis [12, 18] to inter-operator, and optimize it to make our analysis faster than existing methods. We base on observations: 1) Analyzing MA and BR of a tensor can be done easily by traversing the dimension order instead of analyzing tile movement step by step. 2) The dimensions in other operators would not affect MA in this operator, even if they

are fused. 3) Because the tile size in an operator is constant in each execution, liveness analysis for BR computation can be applied at the tensor level instead of the tile. This overlooks tile execution order within an operator, significantly simplifying analysis.

With the symbolic memory access volume (MA) and buffer size requirements (BR), we can filter out inferior or equivalent schedules by comparing them mathematically. For example, in Fig. 4(c), if we mutate the mm_2 dimension order from $(1N, 2L, 3M)$ to $(1N, 2M, 3L)$, although the MA remains the same as before, the BR of C_{ML} changes from $T_L \cdot T_M$ to $L_size \cdot M_size$, where T_{dim} represents the tile size of dimension dim , and dim_size represents the overall size of dimension dim . It's evident that $T_L \cdot T_M \leq L_size \cdot M_size$, although the specific tile size is not known. So, we filter out the mutated one.

4.2 Tiling Size Optimization

From the previous step, we obtain the generated operator schedules along with their corresponding MA and BR formulas. These are associated with variables representing the tile size T_{dim} and tile number N_{dim} for each dimension:

$$MA = \sum_{\text{tensor}, \text{dim}_o}^{\text{unfused tensors}} (\text{tensor_size} \times \begin{cases} 1, & \text{tensor is stationary} \\ N_{\text{dim}_o}, & \text{tensor is non-stationary} \end{cases}) \quad (2)$$

$$BR = \max(\sum_{\text{tensor}}^{\text{live tensors}} ((\prod_{\text{dim}_j}^{\text{buffered dims}} \text{dim}_j_size) \times \prod_{\text{dim}_i}^{\text{unbuffered dims}} T_{\text{dim}_i})) \quad (3)$$

where dim_o is the dimension in another tensor causing repeated memory access. The *unfused tensors*, *stationary*, *live tensors* and *buffered dims* are already determined by the schedule. dim_size is constant. We transform these formulas into a Mixed Integer Programming (MIP) problem to optimize T_{dim} s and N_{dim} s:

minimize : MA

constraints : $BR \leq \text{Buffer Size}, T_{dim} \times N_{dim} \geq dim_size$

We employ the Gurobi optimizer [6] to solve the MIP problem and obtain the minimal MA along with the corresponding tile sizes. By comparing the specific MA s among the superior schedules, we select the best one, determining the optimal tile schedule and size.

4.3 PE Hardware Mapping Optimization

The final step maps the tiles to PEs with local buffer to actually do the computation. Firstly, we utilize a flexible partition strategy [5] to divide each tile into smaller workloads, distributing them across the corresponding PEs. Secondly, for each partition scheme, we employ a search-based mapping strategy [12, 18] to explore the optional mapping on the MACs in a PE over space and time.

Note that the mapping at the tile level is decoupled from the prior two steps. The scheduling step will not affect the mapped efficiency, but the tiling step will. So we have to constraint the tile size in the tiling step, e.g. larger than a threshold to ensure a full spatial mapping in a PE. Regarding different PE architectures, this framework also works with their own mapping tools available.

5 EVALUATION AND RESULTS

5.1 Experiment Setup

Spatial architecture parameters. We evaluate DAT on spatial architectures as illustrated in Fig. 9 using two different configurations, i.e. edge and cloud as shown in Table 2 based on [2, 15] for

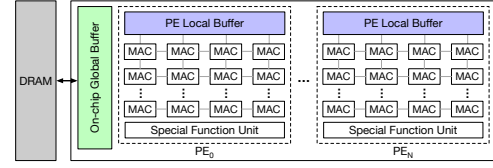


Figure 9: Spatial accelerator architecture modeling.

Table 2: Spatial architecture parameters

Platform	# of PEs	Global Buffer	On-Chip BW	Off-Chip BW
Edge	16	2MB	0.5TB/s	50GB/s
Cloud	512	32MB	4TB/s	400GB/s

Table 3: Transformer model parameters

Model	# of Heads	Seq. Length	Hidden Size
Bert	12	1024	768
GPT-2	12	2048	768
XLNet	16	1024	2048
DeBERTa-v2	24	1024	1536
LLaMA2	32	4096 (128-64K)	4096
ALBERT	64	1024	4096

edge and [8, 9] for cloud. Both configurations share the same PE design, which consists of 256 MACs and a 128KB PE buffer.

Architecture modeling. We adopt MAESTRO [12] to obtain the execution cycle and power consumption of each PE, which is an open-source model commonly used in dataflow optimization works [11, 13, 19]. For memory modeling, we adopt the methodology in FLAT [10]. Shared memory resources are represented with limited bandwidth, enabling the assessment of memory-bound operations and identification of potential bottlenecks related to memory bandwidth utilization within the accelerator. For the energy parameter used in modeling, the MAC and DRAM energy cost are estimated at 0.02 pJ/op and 7 pJ/bit [19]. The power of SRAMs with different capacities are obtained from commercial SRAM compiler.

Attention-based workloads. We study six recent open-source attention-based models. The model sizes and input sequence lengths are shown in Table 3. The batch size is set as 16. We additionally evaluate LLaMA2 with sequence lengths from 128 to 64K.

Comparisons. We compare DAT with several SOTA dataflow explorers on the given spatial accelerators. For intra-operator, we evaluate [7, 11, 18] and choose the best result in terms of performance for each case. For inter-operator, we choose FLAT [10] and TANGRAM [5], which fuse two operators using fixed dataflow options. We evaluate two DAT versions, i.e. DAT-2 and DAT- ∞ . DAT-2 restricts at most two operators fused, and therefore shows the potential of considering all the dataflow options on finding the global optimal fusion on two operators. DAT- ∞ does not restrict the number of operators to be fused, and therefore shows the capability of DAT on multi-operator fusion.

5.2 Experimental Results

Performance. Fig. 10 reports the performance results normalized to the peak FLOPs of the target accelerator, which also indicates the utilization. For edge, DAT- ∞ delivers an average performance improvement of 2.23 $\times$, 2.26 $\times$ and 2.24 $\times$ over INTRA, TRANGRAM, and FLAT, respectively. For cloud, it is 6.74 $\times$, 6.33 $\times$ and 1.74 $\times$.

The improvement mainly comes from that DAT can examine all the possible dataflow options, so it ought to find better solutions.

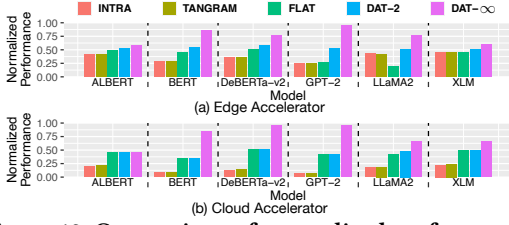


Figure 10: Comparison of normalized performance.

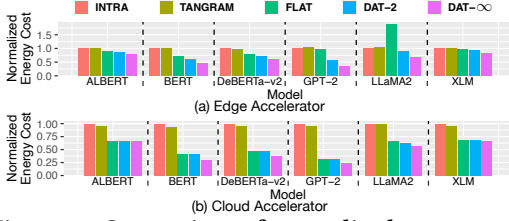


Figure 11: Comparison of normalized energy cost.

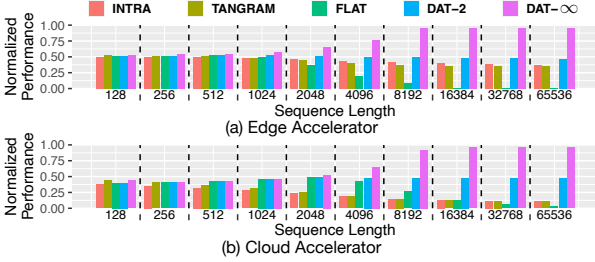


Figure 12: LLaMA2 normalized performance with different sequence length.

Even if two operators are fused, DAT-2's improvement over TANGRAM and FLAT, i.e. $1.58\times$ and $1.57\times$ (edge), $3.70\times$ and $1.03\times$ (cloud), evidently demonstrates this capability. DAT- ∞ enlarges the advantage with $1.43\times$ (edge) and $1.71\times$ (cloud) improvement than DAT-2 by fusing more than two operators, and demonstrates stable improvement even with limited on-chip buffer as in edge accelerator. In contrast, FLAT and TANGRAM applying fixed dataflow fail to fuse more than two operators, or in face of *softmax*.

Energy. Fig. 11 reports the energy cost normalized to INTRA for comparison. For edge, DAT- ∞ achieves an average energy reduction of 38.6%, 38.8% and 35.5% compared to INTRA, TANGRAM and FLAT, respectively. For cloud, it is 53.0%, 51.2% and 15.5%.

Sensitivity to sequence length. Sequence length empowers the attention-based models. The need for long sequences (e.g. 64K) has already gained prominence in the machine learning community [21]. So we vary the sequence length from 128 to 64K in LLaMA2.

Fig. 12 shows that the performance of INTRA and TANGRAM decreases. It is mainly because of the low data reuse in $Q \times K^T$, which dominates the attention computation with longer sequence length. Specifically, FLAT improves at the beginning, but when the sequence becomes longer, its performance drops rapidly because its fusion style stores entire tensor rows on-chip stiffly, causing a substantial buffer requirement. In contrast, DAT-2 and DAT- ∞ improves across all sequence lengths. Compared to DAT-2 which experiences slight performance reduction with 64K sequences length, DAT- ∞ notably performs better and stable. This largely owns to that it turns the negative long sequence challenge on $Q \times K^T$ into an positive performance opportunity, as larger matrix multiplications can lead to higher data reuse, especially when they are fusible.

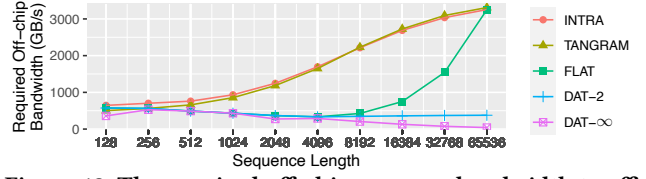


Figure 13: The required off-chip memory bandwidth to efficiently run LLaMA2 on the cloud accelerator.

Minimum memory bandwidth. Fig. 13 reports the off-chip memory bandwidth requirement to achieve high utilization. It shows that INTRA, TANGRAM and FLAT all demand significant memory bandwidth of thousands of GB/s particularly with longer sequences, which is prohibitive. In contrast, DAT addresses this issue by effectively lowering the requirement by 71.3%, 65.2% and 41.1% against INTRA, TANGRAM and FLAT, respectively.

6 CONCLUSION

This paper proposes dataflow adaptive tiling (DAT) to enable efficient fusion of multiple operators connected in any form and chained in any length. Based on DAT, we broaden the dataflow exploration from intra-operator to inter-operator and develop a framework to speed up DAT exploration. The experimental results show that DAT outperforms existing dataflow optimizers particularly on long sequence, leading to significant improvements on performance and energy.

REFERENCES

- [1] Manoj Alwani et al. 2016. Fused-Layer CNN Accelerators. In *IEEE Micro*.
- [2] Yu-Hsin Chen et al. 2019. Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices. In *JETCAS*.
- [3] Tri Dao et al. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*.
- [4] Jacob Devlin et al. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT*.
- [5] Mingyu Gao et al. 2019. TANGRAM: Optimized Coarse-Grained Dataflow for Scalable NN Accelerators. In *ASPLOS*.
- [6] Gurobi Optimization, LLC. 2023. Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
- [7] Qijing Huang et al. 2021. CoSA: Scheduling by Constrained Optimization for Spatial Accelerators. In *ISCA*.
- [8] Norman P. Jouppi et al. 2017. In-Datcenter Performance Analysis of a Tensor Processing Unit. In *ISCA*.
- [9] Norman P. Jouppi et al. 2021. Ten Lessons From Three Generations Shaped Google's TPUv4: Industrial Product. In *ISCA*.
- [10] Sheng-Chun Kao et al. 2023. FLAT: An Optimized Dataflow for Mitigating Attention Bottlenecks. In *ASPLOS*.
- [11] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm. In *ICCAD*.
- [12] Hyoukjun Kwon et al. 2020. MAESTRO: A Data-Centric Approach to Understand Reuse, Performance, and Hardware Cost of DNN Mappings. In *IEEE Micro*.
- [13] Liqiang Lu et al. 2021. TENET: A Framework for Modeling Tensor Dataflow Based on Relation-Centric Notation. In *ISCA*.
- [14] Wei Niu et al. 2021. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion. In *PLDI*.
- [15] Kiran Seshadri et al. 2022. An Evaluation of Edge TPU Accelerators for Convolutional Neural Networks. In *IISWC*.
- [16] Yining Shi et al. 2023. Welder: Scheduling Deep Learning Memory Access via Tile-graph. In *OSDI*.
- [17] Ashish Vaswani et al. 2017. Attention is all you need. In *NeurIPS*.
- [18] Xuan Yang et al. 2020. Interstellar: Using Halide's Scheduling Language to Analyze DNN Accelerators. In *ASPLOS*.
- [19] Shixuan Zheng et al. 2022. Atomic Dataflow Based Graph-Level Workload Orchestration for Scalable DNN Accelerators. In *HPCA*.
- [20] Siz Zheng et al. 2023. Chimera: An Analytical Optimizing Framework for Effective Compute-intensive Operators Fusion. In *HPCA*.
- [21] Haoyi Zhou et al. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI*.